

HW2: TAGE Branch Predictor Design

Lin Yen-Yu
112550098

Abstract—We profile the Aquila core’s bimodal BPU on CoreMark and implement a compact TAGE predictor with practical tweaks: weaker T_0 init, shorter histories, a per-PC statistical corrector, an extra long-history table (T4), and a direction-biased gate for forward/backward behavior. Across these iterations, our final design lowers total misprediction by about 19% versus bimodal while keeping CoreMark throughput essentially unchanged. The results suggest that modest hardware additions and simple control heuristics can capture long-history loops and protect TAGE from forward-branch pollution.

Index Terms—RISC-V, Aquila, branch prediction, TAGE, BHT, CoreMark, performance profiling, FPGA

I. INTRODUCTION

In this assignment, we analyze the branch prediction behavior of the Aquila RISC-V core using the CoreMark benchmark. The goal is to evaluate how the current 2-bit bimodal predictor affects performance, and to collect branch statistics such as hit and miss rates for different branch types. Based on these observations, we outline the initial design of a TAGE branch predictor to improve prediction accuracy and overall CoreMark efficiency.

II. BRANCH PROFILING MECHANISM

The branch profiler in this work is implemented in the **execution stage**, since the signal indicating whether a branch is mispredicted (`exe_branch_misprediction`) is generated at this stage. Therefore, all branch-related statistics are collected based on the EX stage, with an additional signal `pred_ok` introduced to represent whether the prediction is correct.

Furthermore, I compared the branch destination address (`exe_branch_target_addr`) with `exe2mem_pc` to distinguish between **forward** and **backward** branches. This classification may provide insights for future predictor tuning or direction-dependent optimizations.

The profiler maintains the following counters for each branch type:

- Total number of occurrences (`total`)
- Number of BHT hits (`bht_hit`)
- Number of correct predictions (`pred_ok`)
- Number of mispredictions (`misp`)

These statistics enable detailed observation of the current 2-bit bimodal predictor’s behavior and serve as the foundation for later TAGE-based predictor design and analysis.

In addition, the BPU can be enabled or disabled through the `aquila_config.vh` configuration file by commenting or uncommenting the line `'define ENABLE_BRANCH_PREDICTION`. The size of the Branch History Table (BHT) is adjustable in `bpu.v` by modifying

the parameter `ENTRY_NUM = 64`. As described at the bottom of `bpu.v`, the parameter `NBITS` is derived as $\log_2(\text{ENTRY_NUM})$, determining the number of index bits used for BHT access.

III. PROFILING RESULTS

	BPU off	BPU on
Iterations/Sec	98.0197	108.4094

TABLE I: CoreMark performance comparison with and without BPU.

Enabling the BPU improves CoreMark performance from 98.02 to 108.41 iterations per second, representing about a **10.6% increase**. This result indicates that branch prediction effectively reduces pipeline flush penalties and enhances overall instruction throughput.

BHT Size	16 entry	32 entry	64 entry	128 entry	256 entry	512 entry
BHT hit rate (%)	80.47	88.17	93.13	95.21	99.18	99.88
PredACC (%)	98.85	98.78	98.72	98.70	98.71	98.71
Mis Pred Rate (%)	1.14	1.21	1.27	1.29	1.28	1.28

TABLE II: Branch predictor performance under different BHT sizes.

The results in Table II show that increasing the BHT size significantly improves the hit rate—from about 80% at 16 entries to nearly 100% at 512 entries—indicating that larger tables effectively reduce aliasing among branch entries. However, the overall prediction accuracy (PredACC) and misprediction rate remain nearly constant once the BHT exceeds 64 entries, suggesting that the 2-bit bimodal predictor quickly reaches its performance limit.

BHT Size	16 entry	32 entry	64 entry	128 entry	256 entry	512 entry
Fwd HR (%)	89.49	93.16	96.69	97.87	99.49	99.96
Bwd HR (%)	74.33	84.56	90.49	93.21	98.95	99.81
Fwd Acc (%)	98.62	98.53	98.46	98.45	98.47	98.47
Bwd Acc (%)	99.01	98.96	98.91	98.89	98.89	98.89

TABLE III: Forward and backward branch performance under different BHT sizes.

As shown in Table III, both forward and backward branches benefit from a larger BHT size, with hit rates improving rapidly as the table grows. The improvement is more pronounced for backward branches, which are typically associated with loop structures that repeatedly access the same branch entries. This indicates that expanding the BHT effectively captures these recurring patterns and reduces aliasing. However, similar to the overall trend observed earlier, the prediction accuracy (Acc) remains relatively stable beyond 64 entries, suggesting that further enhancement requires a more sophisticated predictor such as TAGE to exploit longer history correlations.

IV. TAGE BPU IMPLEMENTATION

A. Brief Introduction of TAGE BPU

The TAGE predictor consists of multiple prediction tables, typically denoted as $T_0, T_1, T_2, \dots, T_n$, where each table corresponds to a different history length. The base predictor T_0 is a simple bimodal table indexed only by the program counter (PC) and does not use tags. It serves as the default prediction source when no longer-history table provides a confident match.

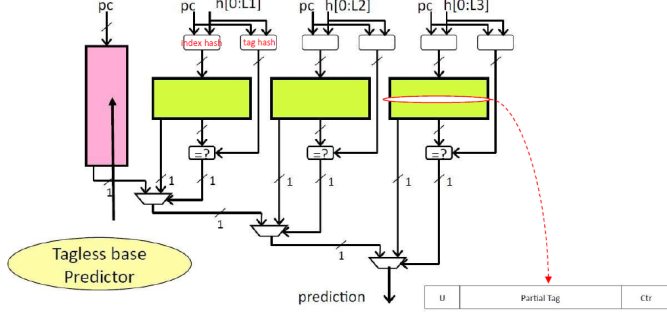


Fig. 1: TAGE Predictor

For each table T_i ($i > 0$), an entry stores the following fields:

- A **partial tag** derived from the PC and the folded global history.
- A **prediction counter** representing the branch tendency.
- A **usefulness counter** that indicates whether this entry has been useful in past predictions.

Each table T_i has a fixed number of entries, and its index is computed by hashing the PC together with a folded version of the global history of length L_i , where L_i grows exponentially with i (e.g., 4, 8, 16, 32, 64, ...).

When predicting, the BPU searches from the longest-history table down to T_0 . The first table whose tag matches the current PC-derived tag is chosen as the **provider table**, and its counter determines the prediction direction. If no tag matches are found, T_0 is used as the fallback predictor.

During update, the chosen table's counter and usefulness bits are updated based on the correctness of the prediction. If a misprediction occurs, the predictor may allocate new entries in deeper tables using the current global history, ensuring that longer histories gradually learn the correlation between distant branches.

B. Implementation Details

I started by implementing a **3-table TAGE predictor plus the base table T_0 (bimodal)**. Below are the main modifications I made compared to the original bimodal BPU:

1) Modified `branch_hit_o` and `branch_decision_o`

In the original bimodal version, a branch was considered a hit when *the BHT tag matched the PC* (`BHT tag == pc\_i`). In my TAGE version, a hit is defined when *any of the tagged tables (T_1 – T_3) match*. The base table T_0 is only used as a fallback and is **not counted as a hit**. The output

`branch_decision_o` is determined by the **MSB of the 3-bit counter** from the selected TAGE table; if no table matches, the prediction falls back to T_0 . I also changed the write-enable (`we`) logic — in TAGE, I no longer wait for a BHT hit before writing. Instead, I set:

```
we = ~stall & (is_cond_branch | is_jal);
```

This ensures that the branch target address can be written into the BHT as soon as possible.

2) Architecture Design

a) **Table structure** My predictor includes four tables:

- **T_0 (Base):** Direct-mapped table with 3-bit saturating counters (basically an enhanced bimodal predictor).
- **T_1 – T_3 (Tagged tables):** Each table contains:
 - `tag[ENTRY_NUM]` (8-bit tags)
 - `counter[ENTRY_NUM]` (3-bit saturating counters)
 - `useful[ENTRY_NUM]` (1-bit flags used for replacement and allocation)

b) **Global History Register (GHR)** The global history length is set to `MAX_HIST_LEN`, equal to the history length of T_3 . On every *Execute stage*, if the instruction is a conditional branch, the history shifts in as `{history[62:0], taken}`. I also added a separate `global_history_r` register so that signals from the Decode stage stay aligned with the Execute pipeline stage.

c) **Hashing Function** According to André Seznec's *A New Case for the TAGE Branch Predictor*, the hash length should increase with the table's history length, and the index should be derived using `PC XOR GHR`. Therefore, I compute my hash as:

```
pc[TAG_WIDTH+1:2] ^
(history[HIST_LEN_2-1:0] ^
history[TAG_WIDTH-1:0]);
```

This captures both PC bits and a portion of the global history for better table diversity.

3) TAGE Profiler

I modified the BPU's I/O so that it outputs `provider_sel_r` to `core_top.v`. Using a similar profiling technique from Lab 1, I counted how often each table (T_0 – T_3) was selected among all branches. This helps me understand the distribution of provider usage and analyze how TAGE behaves in practice, so I can tune the history lengths and replacement policy later for better accuracy.

V. TAGE BPU RESULT ANALYSIS AND IMPROVEMENT

	bimodal	TAGE	TAGE with WT init
bwd_misp_rate (%)	1.0887	1.0373	0.6884
fwd_misp_rate (%)	1.5322	3.4000	2.8201
total_misp_rate (%)	1.2777	1.9754	1.5481
total_inc/dec_rate (%)	0.00	+54.61	+21.16
Iterations/Sec	109.90	108.51	108.48
Total time (secs)	18.198	18.432	18.436

TABLE IV: Comparison of performance and misprediction rates among bimodal, TAGE, and WT-initialized predictors.

Predictor	T0(%)	T1(%)	T2(%)	T3(%)
bimodal	100	—	—	—
TAGE	67.1187	18.6694	13.3589	0.8530
WT init	59.4571	26.9393	8.7774	4.8263

TABLE V: Table match ratio comparison among bimodal, TAGE, and WT-initialized predictors.

For my initial 3-table TAGE implementation, both the forward and backward branch misprediction rates were higher than those of the bimodal predictor. I suspect this was because the T_0 table was initialized to **Strongly Not Taken**, which doesn't work well for forward branches that are frequently taken. So, I changed the initialization to **Weakly Not Taken** to make it adapt better to forward branches. After this modification, I noticed that the backward branch misprediction rate dropped significantly, which means the real bottleneck of my TAGE design is actually **the forward branches**.

	bimodal	WT init	WT init + short his + SC
bwd_misp(%)	1.0887	0.6884	1.0395
fwd_misp(%)	1.5322	2.8201	1.5284
total_misp(%)	1.2777	1.5481	1.2479
total_inc/dec(%)	0.00	+21.16	-2.33
Iterations/Sec	109.90	108.48	108.48
Total time (secs)	18.198	18.436	18.436

TABLE VI: Comparison of bimodal, TAGE with WT initialization, and TAGE with WT initialization + short history + SC.

Predictor	T0(%)	T1(%)	T2(%)	T3(%)
bimodal	100	—	—	—
WT init + short his + SC	99.6570	0.3122	0.0304	0.0004

TABLE VII: Table match count comparison between bimodal and the WT-initialized TAGE with shorter history and Statistical Corrector (SC).

To improve forward-branch accuracy, I first shortened the geometric history lengths from **4-16-64** to **4-8-32**. Then, following Ref. [2], I added a simple **Statistical Corrector (SC)**: each PC gets a 2-bit *confidence counter* that decides whether to trust the bimodal (T_0) or the tagged TAGE path.

Update rule (per branch at that PC):

- TAGE *success* $\Rightarrow$ confidence ++
- TAGE *fail* $\Rightarrow$ confidence --
- bimodal *success* $\Rightarrow$ confidence --
- bimodal *fail* $\Rightarrow$ confidence ++

I initialize each PC's confidence to $2'b01$ (lean toward bimodal). We only start considering TAGE after the bimodal path fails at that PC.

This configuration *did* reduce the overall misprediction rate below the bimodal baseline. However, from the table *match-rate* breakdown, TAGE+ST behaved very similar to bimodal: most forward branches still fell back to T_0 , so the ST counter was rarely exercised there. My guess is that (i) we do not have enough tagged tables to better capture backward-branch correlations, and (ii) forward branches didn't benefit much from ST (confidence stayed near the bimodal side), making the final behavior largely T_0 -driven.

	bimodal	Adding T4 + direction biased
bwd_misp (%)	1.0887	0.3564
fwd_misp (%)	1.5322	2.0171
total_misp (%)	1.2777	1.0401
total_inc/dec (%)	0.00	-18.60
Iterations/Sec	109.90	109.24
Total time (secs)	18.198	18.308

TABLE VIII: Comparison of bimodal and the final TAGE variant (WT init + short history + ST + T4 + direction bias).

Predictor	T0(%)	T1(%)	T2(%)	T3(%)	T4(%)
bimodal	100	—	—	—	—
T4 + direction biased	87.4180	11.6665	0.5957	0.1646	0.1552

TABLE IX: Table match count comparison between bimodal and the final TAGE variant (with T4 and direction bias).

To further improve prediction accuracy, I added a new **Table 4** dedicated to backward branches, which corresponds to a **history length of 128**. For forward branches, I introduced a **direction-biased mechanism**, similar in spirit to the *Statistical Corrector Predictor (ST)*. Each PC is assigned a 2-bit counter that tracks whether the branch tends to behave as a forward or backward branch. The idea is to dynamically choose between the bimodal predictor and TAGE depending on the branch direction.

Direction-biased logic:

- If the counter for a PC is $2'b00$, the branch is treated as *forward*, and the prediction is forced to use the **bimodal** predictor.
- Otherwise, the predictor relies on the existing **confidence counter** to decide whether to use TAGE.

This approach provides a buffer zone for *mixed-direction branches*—those that sometimes act like forward and sometimes backward—while allowing consistently forward branches to stay on bimodal, preventing them from polluting the TAGE tables.

This version achieved extremely high accuracy on backward branches and also maintained solid accuracy on forward branches. As a result, the overall **total misprediction count** dropped by about **19% compared to the bimodal predictor**.

REFERENCES

- [1] <https://jilp.org/vol8/v8paper1.pdf>
- [2] <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7851464>
- [3] https://blog.csdn.net/weixin_45171433/article/details/132729757
- [4] <https://zhuanlan.zhihu.com/p/660228762>